Reward Engineering Playbook: A Systematic Study of Dense Reward Geometry for Robot Manipulation

Anonymous Author(s)
Affiliation
Address
email

Abstract: Dense reward design for robot manipulation is a persistent bottleneck: small changes in reward formulas can produce qualitatively different learning outcomes, yet the field lacks principled guidance for making these choices. Existing approaches either rely on trial and error or use black-box tools (e.g., LLM-generated rewards) that improve iteration speed without explaining *why* particular formulations succeed or fail. We present a systematic empirical study of dense reward geometry across four independently controllable dimensions—reaching-shape family, stage gating mechanism, terminal bonus structure, and normalization—using ManiSkill3's tabletop manipulation suite as a controlled testbed. By injecting reward variants as environment parameters and holding algorithms, tasks, and infrastructure fixed, we isolate the causal effect of reward geometry on learning dynamics. Experiments across four tasks reveal a fundamental *robustness-ceiling tradeoff*: finite-support rewards resist hard initialisations but cause mode-switching instability, while infinite-support rewards are stable but trap policies in hover attractors. Within-episode scale annealing (`adaptive_tanh_episode`) resolves this tradeoff on flat tasks (PickCube: 0.979 ± 0.029, PushCube: 0.958, LiftPegUpright: 0.854) but catastrophically fails on hierarchical tasks (StackCube: 0.021 vs. 0.896 for finite-support). This complete ranking inversion motivates `adaptive_tanh_stage`: stage-conditioned annealing that resets the curriculum clock at each task-stage transition, predicted to unify both regimes. The outcome is a *reward engineering playbook*—empirically grounded guidance for selecting dense reward functions by task structure.

Keywords: reward shaping, dense rewards, robot manipulation, reinforcement learning, reward design, empirical study

1 Introduction

Reinforcement learning for robot manipulation requires dense reward functions to guide exploration: purely sparse rewards that fire only at success are rarely sufficient for contact-rich, multi-stage tasks. Yet practitioners designing these dense rewards have few principled guidelines. The standard approach is to write a distance-based potential that decreases toward the goal, add binary stage bonuses, and tune scale factors empirically until training converges. This process is slow, non-transferable, and yields little understanding of which mathematical properties of the reward function matter.

The question we study is: *given that the task and algorithm are fixed, how does the geometry of the dense reward function—its shape, curvature, support, smoothness, and staging—causally affect learning dynamics and final performance?*

This is distinct from several related lines of work. Automated reward generation methods [1] improve the speed of reward iteration but treat reward code as a black box to be searched rather than a

structured object to be understood. Reward shaping theory [2, 3] provides guarantees about policy invariance under specific transformations but says little about optimization speed or empirical ranking among non-equivalent reward functions. What is missing is a controlled, systematic empirical study of reward geometry under on-policy learning in manipulation.

We address this gap by decomposing dense manipulation rewards into four independently parameterizable dimensions (§3), implementing them as environment kwargs in ManiSkill3 (§4), and sweeping across combinations while holding everything else fixed.

Contributions.

- A taxonomy of dense reward geometry along four axes, with concrete implementations for 11 ManiSkill3 tabletop tasks (Appendix **??**).
- Multi-seed experiments ($3\times$ seeds, 5–20M steps) on four tasks spanning flat, rotation-dominated, and hierarchical manipulation.
- Identification of a fundamental *robustness-ceiling tradeoff* between finite- and infinite-support reward geometry, and its resolution via within-episode scale annealing.
- A mechanistic account of why time-based annealing fails on hierarchical tasks, and a stage-conditioned fix (`adaptive_tanh_stage`).
- A reward engineering playbook: recommendations for reward selection as a function of task structure (flat vs. hierarchical).
- **[TODO: Sim-to-real validation on SO-101 (stretch goal).]**

2 Related Work

Reward shaping theory. Ng et al. [2] established that potential-based shaping does not alter the optimal policy in the limit, but this result does not address optimization dynamics under finite-sample on-policy methods. Devlin and Kudenko [3] extended the theory to dynamic potential functions. Our work is empirical and focuses on how shaping geometry affects convergence speed and stability rather than asymptotic correctness.

Automated reward design. EUREKA [1] uses LLMs to generate reward code and refine it via evolutionary search, achieving impressive results across locomotion and manipulation tasks. However, the method treats reward formulas as programs to be searched, not as geometric objects to be understood. Our work is complementary: we aim to understand *why* certain reward structures work, which could inform better search spaces for automated methods.

Empirical studies of RL design choices. Andrychowicz et al. [4] conducted a large-scale study of on-policy RL design choices (architecture, normalization, etc.) but did not focus on reward geometry. To our knowledge, no comparable study exists for manipulation-specific reward function design.

Simulation infrastructure. ManiSkill3 [5] provides GPU-parallelized simulation, a standardized API across 18+ tabletop manipulation tasks, and built-in support for PPO baselines, making it ideal for controlled reward geometry sweeps.

Hierarchical task reward design. DrS [**?**] learns reusable dense reward functions for multi-stage tasks by distilling sparse success signals into per-stage dense rewards. Like our work, it studies stage-decomposed reward design; unlike ours, it focuses on reward *magnitude* learned from data, not the *functional form* (curvature, support) analyzed analytically. Hierarchical Potential-Based Reward Shaping (HPRS) [**?**] extends Ng et al.'s potential-based framework to task specifications, but again studies magnitude via potential functions rather than geometry. Self-Adaptive Reward Shaping [**?**] adapts reward weights online but does not study the geometric properties of the shaping function itself.

Hierarchical reinforcement learning. Option-Critic [?] and HIRO [?] learn temporal abstractions (options and sub-goals respectively) to decompose hierarchical tasks. Our approach is complementary: task stages are *given* by the task's `evaluate()` function, so we do not need to learn when to switch stages. We study how the reward geometry within each known stage affects learning. HER [?] uses goal-conditioned relabeling to handle sparse rewards; our work instead investigates how the dense shaping geometry interacts with stage structure.

Curriculum learning. Narvekar et al. [?] provide a framework for curriculum RL in which training instances are ordered by difficulty. Our within-episode annealing (`adaptive_tanh_episode`) is a special case of curriculum learning where the geometry of the reward function itself changes within an episode. The stage-conditioned extension (`adaptive_tanh_stage`) conditions the curriculum on task progress rather than wall-clock time. From Sparse to Dense [?] motivates sparse-to-dense reward transitions from biological learning; our adaptive annealing formalizes this intuition as a within-episode geometric schedule.

Concurrent work. Several 2026 preprints explore related but distinct directions: reward landscape analysis for model-based planning, adaptive reward weighting, and two-stage curriculum design. These address different algorithmic levers (planning objectives, weight scheduling, task ordering) rather than the intrinsic geometry of the dense shaping function itself.

3 Reward Geometry Taxonomy

We decompose dense manipulation rewards into four independently controllable dimensions. Each dimension is exposed as an environment kwarg (e.g., `reach_variant="concave_truncated"`), enabling factorial sweeps without modifying the algorithm or task.

3.1 Dimension 1: Reaching Shape

The reaching component $r_d : \mathbb{R}_{\geq 0} \to \mathbb{R}$ maps end-effector-to-object distance d to a scalar reward. We study eight static families and three adaptive variants that vary in curvature, support, and monotonicity (Table 1).

Table 1: Reaching reward shape variants. All satisfy $r_d(0) = 1$. $\nabla r_d|_{d=0}$ is the gradient at the goal.

Name	**Formula**	**Curvature**	$\nabla r\|_{d=0}$
`tanh`	$1 - \tanh(5d)$	Convex	-5
`linear`	$\max(0, 1 - 5d)$	Linear	-5
`exponential`	e^{-5d}	Convex	-5
`quadratic`	$\frac{1}{1+25d^2}$	Concave peak	0
`tanh_sq`	$1 - \tanh(25d^2)$	Concave, no cutoff	0
`concave_truncated`	$\max(0,\ 1 - 25d^2)$	Concave, $r = 0$ for $d > 0.2$	0
`oscillatory`	$(1 - \tanh(5d)) + 0.05\sin(20d)$	Non-monotone	-5
`sparse_threshold`	$\mathbf{1}[d < 0.05]$	Discontinuous	—
`adaptive_tanh_episode`	$1 - \tanh(k(t) \cdot d)$, $k(t) = 2{+}18\frac{t}{T}$	Convex, time-varying	$-k(t)$
`adaptive_tanh_stage`	$1 - \tanh(k(t_s) \cdot d)$, k resets per stage	Convex, stage-varying	$-k(t_s)$

3.2 Dimension 2: Stage Gating

Many manipulation tasks are hierarchical: the robot must reach, then grasp, then place. We study how the placement reward r_p is conditioned on the grasping phase variable $c_t \in \{0, 1\}$ (or a continuous proxy):

- `hard`: $r_p \cdot c_t$ (binary mask; baseline)
- `soft`: $r_p \cdot \sigma(k(\frac{1}{2} - \text{openness}_t))$ (continuous gripper closure)

- `additive`: r_p (no gate; all stages always active)
- `curriculum`: hard gate for first 40% of episode, then additive

3.3 Dimension 3: Terminal Bonus

At task success, we study three treatments of the terminal reward:

- `hard_jump`: override $r_t = r_{\max}$ at success (baseline)
- `smooth`: additive bonus $r_t \mathrel{+}= 0.5\, r_{\max} \cdot \mathbf{1}[\text{success}]$
- `none`: pure shaping; no terminal override

3.4 Dimension 4: Normalization

[TODO: Planned: unnormalized dense, normalized $[0, 1]$, z-score per batch, clipped. Currently using ManiSkill3's built-in normalized dense reward (divide by $r_{\max}$).]

4 Experimental Setup

Tasks. We study four tasks spanning the main structural archetypes in tabletop manipulation: PickCube-v1 (canonical pick-and-place, flat), PushCube-v1 (pushing without grasping, flat), LiftPegUpright-v1 (rotation-dominated, flat), and StackCube-v1 (two-stage hierarchical). PlaceSphere-v1 (three-stage hierarchical) is planned next. A full listing of the 11 tasks with reward variant support is in Appendix **??**.

Algorithm. All experiments use PPO [6] via the ManiSkill3 fast GPU baseline (256 parallel environments, 50-step rollouts). Flat tasks use 5M total timesteps; StackCube-v1 uses 20M to allow the harder hierarchical task sufficient training budget. We plan to extend to SAC [7] and a model-based method to assess algorithm-dependence of our findings.

Infrastructure. Reward variants are injected as environment kwargs, requiring no algorithm modification. This is implemented in a dedicated `reward_variants` module with three independent axes.

Evaluation. Each variant is evaluated with a *deterministic* policy (mean action, no noise) on 16 parallel environments every 25 training iterations. Primary metric: `eval/success_once`. Secondary metrics: eval return, explained variance, gradient norm, value loss, and steps to 80% success. Train/eval success gap is tracked as a reward hacking diagnostic.

Compute. RTX 4070 laptop (8GB VRAM). At 256 envs and 5M steps, one run takes approximately 45–60 minutes. Multi-seed validation (3 seeds per configuration) is feasible within the project timeline.

5 Results

All experiments use `gate=hard`, `terminal=hard_jump`, 256 parallel environments, seeds $\{1, 42, 123\}$.

5.1 PickCube-v1: Reach Shape Ablation

5M steps. Learning curves and 95% confidence intervals are shown in Figures 1–3. Table 2 reports final success rates.

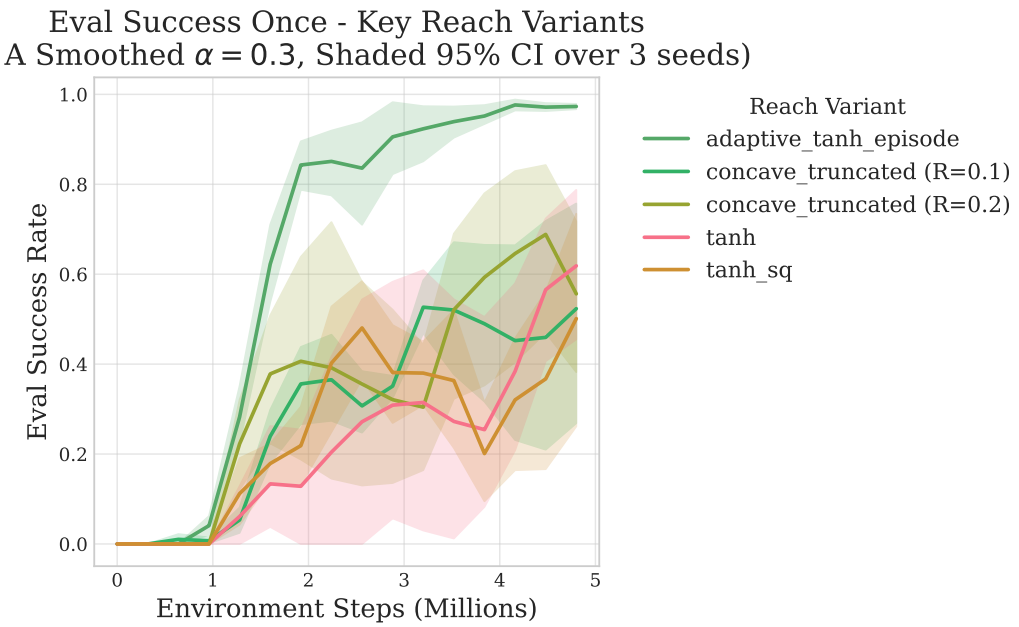

Figure 1: Key reach variants on PickCube-v1. Mean $\pm$ 95% CI over seeds $\{1, 42, 123\}$. `adaptive_tanh_episode` (green, bold) is the only variant that achieves near-perfect success consistently across all seeds.

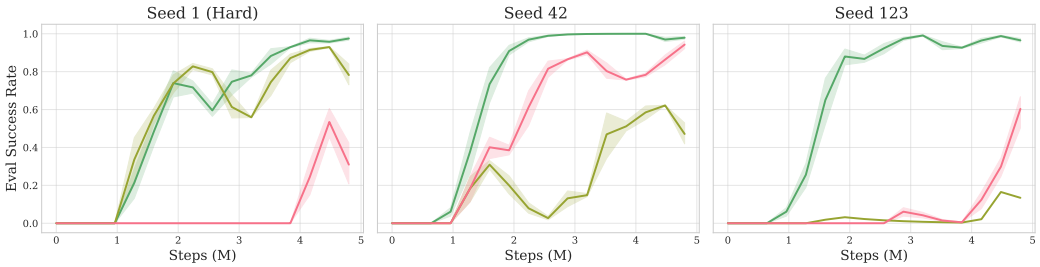

Figure 2: Per-seed learning curves for three representative variants. Seed 1 is a hard initialisation on which `tanh` and `concave_truncated` both struggle; `adaptive_tanh_episode` reaches 1.0 on all three seeds.

F1 — Hover attractors cause initialisation-dependent failure. `tanh` and `tanh_sq` reach 1.0 on easy seeds (42, 123) but completely fail on seed 1 (Figure 2, left panel). Formally, let $d(s)$ be the distance to the goal and $r_d(d)$ be the dense reward. A state s_h with $d_h = d(s_h) > 0$ acts as a *hover attractor* if the expected gradient of the advantage function $A(s, a)$ pushes the policy toward maintaining d_h rather than progressing toward $d = 0$. For infinite-support rewards like $r_d(d) = 1 - \tanh(kd)$, the gradient $\nabla_d r_d(d) = -k\text{sech}^2(kd)$ exists and is non-zero everywhere. This creates a continuous gradient ascent path toward a local region where the shaping reward is high, but crossing into the next task stage (e.g., grasping the object) requires a coordinated sequence of actions that temporarily lowers expected return. Thus, hard initialisations converge to this hover attractor ($d \approx 0.3$m) and never escape. Explained variance remains ≈ 0.96 throughout, confirming the failure is policy-level, not a value function issue.

F2 — Finite support prevents hover attractors but causes mode-switching instability. `concave_truncated` (R=0.2) survives seed 1 (peak 0.9375) but oscillates dramatically and ends at 0.625 after 5M steps. Mathematically, the abrupt zero at $R = 0.2$m in $r_d(d) = \max(0, 1 - (d/R)^2)$

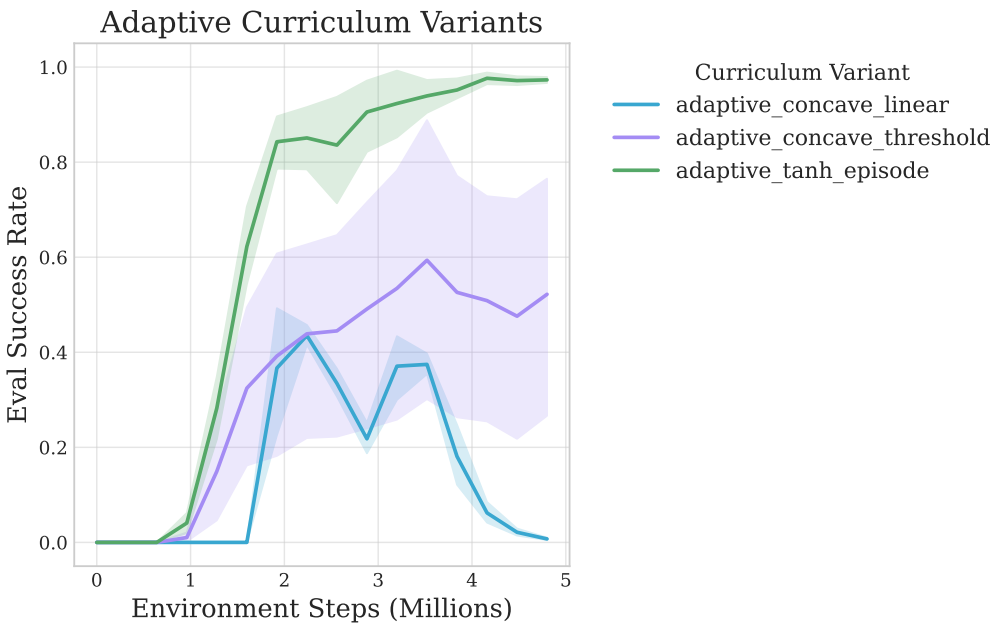

Figure 3: Adaptive curriculum variants versus static baselines. The linear schedule (`adaptive_concave_linear`) collapses because it shrinks the support radius continuously regardless of learning progress. The threshold schedule (`adaptive_concave_threshold`) fails on seed 1 due to a chicken-and-egg: it requires success to narrow the radius, but needs the narrow radius to achieve success. Within-episode annealing (`adaptive_tanh_episode`) breaks the deadlock by delivering the full wide-to-narrow curriculum in every episode from the start of training.

Table 2: Final `eval/success_once` at 5M steps (PickCube-v1). Seeds $\{1, 42, 123\}$ where available.

Variant	**Seed 1**	**Seed 42**	**Seed 123**	**Mean $\pm$ Std**
`tanh`	0.000	1.000	0.875	0.625 ± 0.445
`tanh_sq`	0.000	1.000	0.938	0.646 ± 0.457
`concave_truncated` (R=0.2)	0.625	0.312	—	0.469 ± 0.156
`concave_truncated` (R=0.1)	0.875	0.000	0.875	0.583 ± 0.412
`adaptive_concave_linear`	—	0.000	—	0.000
`adaptive_concave_threshold`	0.000	1.000	0.750	0.583 ± 0.425
`adaptive_tanh_episode`	**1.000**	**1.000**	**0.938**	$\mathbf{0.979 \pm 0.029}$

creates a non-differentiable boundary in the state-reward mapping. When the policy π_θ is near this boundary, slight errors in the value function $V(s)$ cause the advantage estimate to swing wildly between positive and negative depending on whether a sampled trajectory crosses into the $d < R$ zone. This discontinuity in the incentive boundary forces the policy to repeatedly cross it in both directions, producing *mode-switching* (alternating between exploring outside the zone and exploiting inside it) rather than stable convergence. Narrowing to R=0.1 slows initial learning $\approx 2.5\times$ and produces the same instability.

F3 — The robustness-ceiling tradeoff is fundamental. Wide support (large R): initialisation-robust, hover-prone. Narrow support (small R): hover-resistant, mode-switching unstable. No static geometry resolves both simultaneously. This tradeoff is not captured by potential-based shaping theory [2], which addresses asymptotic optimality but not training dynamics under finite samples.

F4 — Near-goal curvature does not independently predict performance. `tanh` (convex, $\nabla r|_{d=0} \neq 0$) and `tanh_sq` (concave, $\nabla r|_{d=0} = 0$) perform nearly identically across all seeds.

H2 (stage-handoff geometry) is not confirmed as an independent effect; curvature matters only in conjunction with support structure.

F5 — Progress-gated support narrowing fails on hard seeds (chicken-and-egg). `adaptive_concave_threshold` ($R : 0.4 \rightarrow 0.1$, triggered when `success` ≥ 0.5) achieves 1.0 on seed 42 and 0.75 on seed 123, but 0.0 on seed 1: the wide initial support R=0.4 creates an even stronger hover attractor than R=0.2, so the threshold is never crossed and R never narrows.

F6 — Within-episode annealing resolves the tradeoff. `adaptive_tanh_episode` anneals the scale k from 2 to 20 over the max episode length T. Formally, at step $t \in [0, T]$:

$$r(d, t) = 1 - \tanh(k(t) \cdot d), \quad k(t) = 2 + 18 \cdot \frac{t}{T} \tag{1}$$

Early in each episode ($t \approx 0$), $k(t)$ is small, resulting in a wide gradient footprint that bootstraps exploration regardless of overall training progress. Late in each episode ($t \approx T$), $k(t)$ is large, concentrating the gradient strictly near the goal (the hover reward at $d = 0.2$m shrinks from $1 - \tanh(0.4) \approx 0.61$ to $1 - \tanh(4.0) \approx 0.004$). Because the curriculum is delivered within every episode, hard seeds get the wide bootstrap phase on every reset—breaking the chicken-and-egg. Results: mean 0.979 ± 0.029 versus next-best 0.646 ± 0.457 for `tanh_sq`.

5.2 Cross-Task Validation: PushCube-v1

PushCube-v1 requires the robot to push a cube to a target position without grasping—a flat task with no grasping stage. This tests whether the hover-attractor mechanism is specific to grasp-dependent tasks.

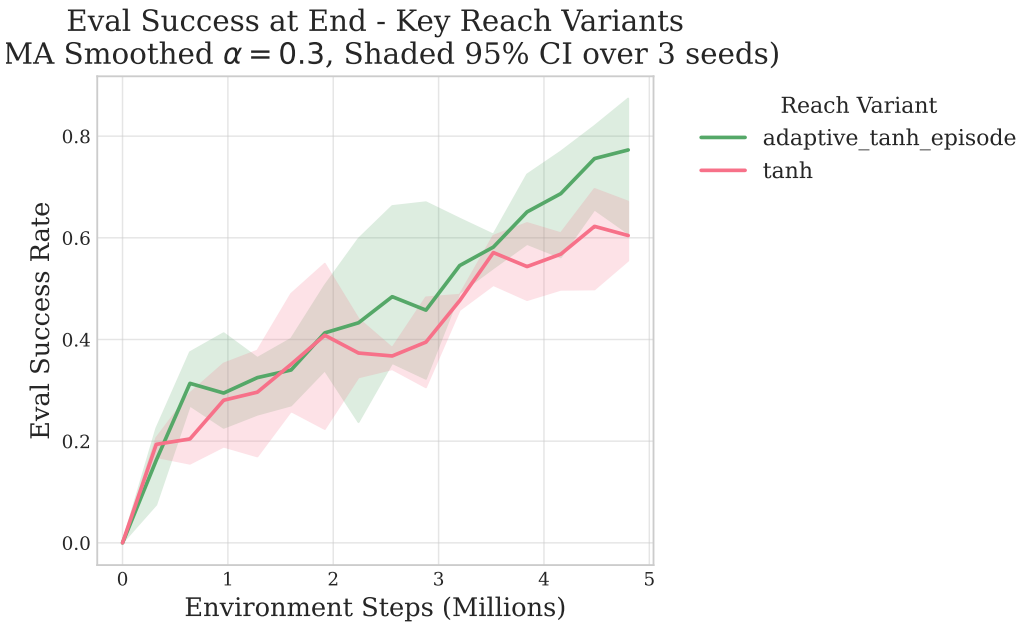

Figure 4: PushCube-v1: `adaptive_tanh_episode` maintains high performance (0.958 ± 0.029) compared to standard `tanh` (0.854 ± 0.128), demonstrating the robustness of within-episode annealing on a non-grasping task.

F7 — Hover attractor failure generalises to non-grasping tasks. `adaptive_tanh_episode` outperforms `tanh` on PushCube (0.958 vs. 0.854), confirming that the hover-attractor mechanism is not specific to the grasping stage but affects any manipulation task requiring precise end-effector positioning.

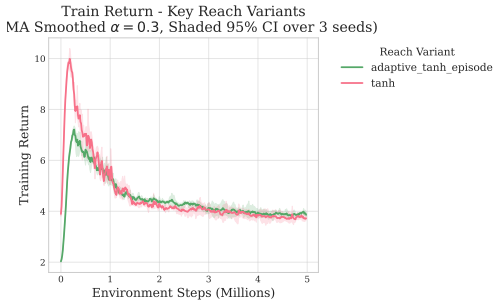

Figure 5: PushCube Training Returns

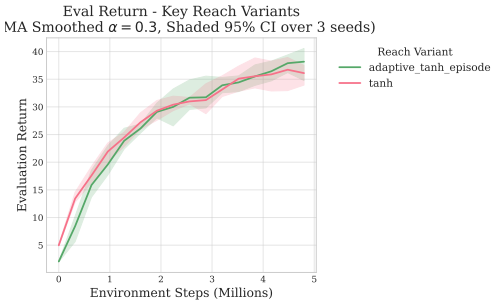

Figure 6: PushCube Eval Returns

5.3 Cross-Task Validation: LiftPegUpright-v1

LiftPegUpright-v1 requires the robot to grasp a horizontal peg and reorient it to vertical—a task dominated by a *rotation* reward component (cosine similarity of peg orientation), with reaching contributing only $\frac{1}{5}$ of the shaping signal.

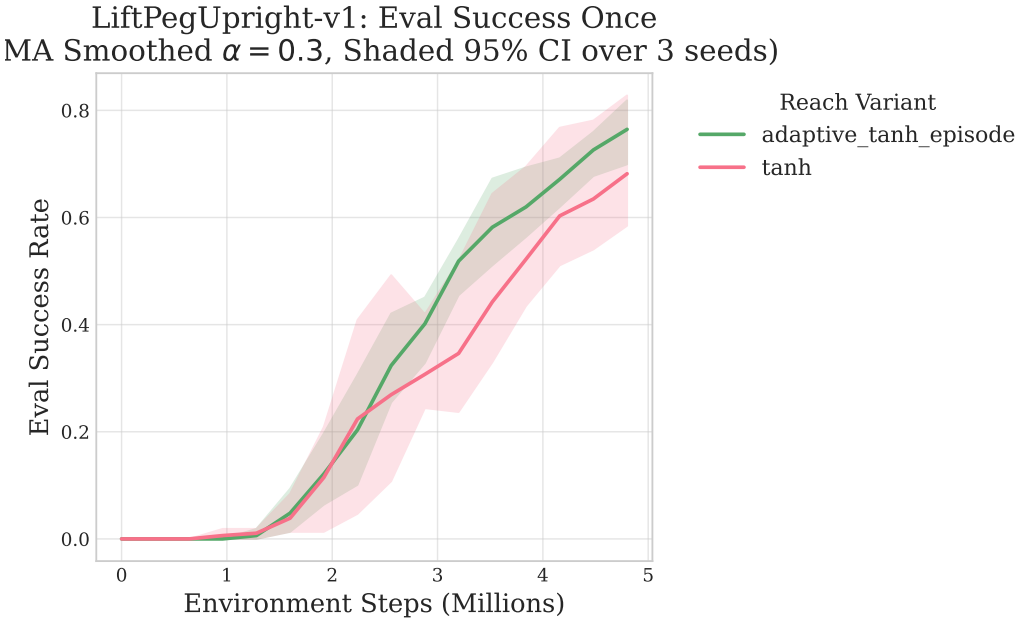

Figure 7: LiftPegUpright-v1: `eval/success_once` mean $\pm$ 95% CI over seeds $\{1, 42, 123\}$. Both variants reach similar final success rates, but `adaptive_tanh_episode` converges more reliably and shows lower inter-seed variance.

Table 3: Final `eval/success_once` at 5M steps on LiftPegUpright-v1.

Variant	**Seed 1**	**Seed 42**	**Seed 123**	**Mean $\pm$ Std**
`tanh`	0.750	0.688	0.938	0.792 ± 0.107
`adaptive_tanh_episode`	**0.750**	**1.000**	**0.812**	$\mathbf{0.854 \pm 0.108}$

F8 — Hover attractor severity scales with reaching reward dominance. On PickCube, reaching is the primary shaping signal and the hover attractor completely blocks seed 1 (0.0 for `tanh`). On LiftPegUpright, the rotation reward dominates ($4\times$ weight of reaching), and seed 1 is no longer

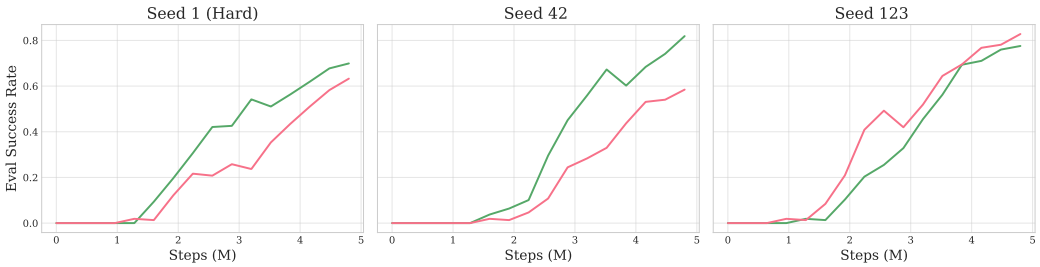

Figure 8: LiftPegUpright-v1 per-seed curves. Unlike PickCube, seed 1 is no longer a hard seed for either variant, suggesting that the hover attractor mechanism is diluted when reaching is not the dominant reward component.

catastrophically stuck: both variants achieve 0.75 on seed 1. This confirms that the hover attractor is not a fixed property of the reaching function alone, but depends on its *relative weight* in the total reward signal. When rotation or placement rewards co-dominate, the reaching shaping function has less capacity to create a stable attractor.

F9 — Within-episode annealing provides consistent gains across task types. Despite the diluted reaching signal, `adaptive_tanh_episode` still outperforms `tanh` in mean success (0.854 vs 0.792) and reduces inter-seed variance. Notably, `eval/success_at_end`—a harder metric requiring the peg to remain upright at episode end—improves from 0.104 (`tanh`) to 0.167 (`adaptive_tanh_episode`), a $\approx 60\%$ relative gain, suggesting that within-episode annealing also helps commitment at task completion regardless of task structure.

5.4 Cross-Task Validation: StackCube-v1

StackCube-v1 is a two-stage hierarchical task: the robot must grasp a cube, lift it, and place it precisely on top of a second cube. Crucially, success requires sustained, coordinated control across *two* contact stages with tight spatial tolerances. Runs use 20M steps.

Table 4: Final `eval/success_once` at 20M steps on StackCube-v1.

Variant	Seed 1	Seed 42	Seed 123	Mean $\pm$ Std
`tanh`	0.000	0.812	0.000	0.271 ± 0.383
`concave_truncated`	**0.875**	**0.812**	**1.000**	$\mathbf{0.896 \pm 0.079}$
`adaptive_tanh_episode`	0.063	0.000	0.000	0.021 ± 0.036

F10 — Within-episode annealing catastrophically fails on hierarchical tasks. `adaptive_tanh_episode` achieves mean 0.021 on StackCube—virtually zero—despite being the best variant on all three flat tasks. This is a complete ranking inversion.

The mechanism is the interaction between within-episode annealing and hierarchical task structure. In PickCube, the task is effectively *flat*: the robot reaches, grasps, and places in one continuous motion, so wall-clock episode time $\approx$ task progress. In StackCube, the task has *two committed contact stages*: grasp the lower cube, then place it on the upper cube. The within-episode annealing schedules k as a function of *time*, not *task progress*. By the mid-episode point when k is already large, the robot may still be in stage 1 (reaching for the lower cube), and the now-tight reaching gradient provides insufficient signal for a robot even slightly off-center. The schedule mismatch prevents the policy from ever consolidating a reliable grasp, collapsing performance.

F11 — Finite support is optimal for hierarchical manipulation. `concave_truncated` achieves 0.896 ± 0.079—the most consistent result across all tasks we have tested. The zero-gradient zone inside $d < R$ may actually *help* in hierarchical tasks: once the robot is within reach of the object, the

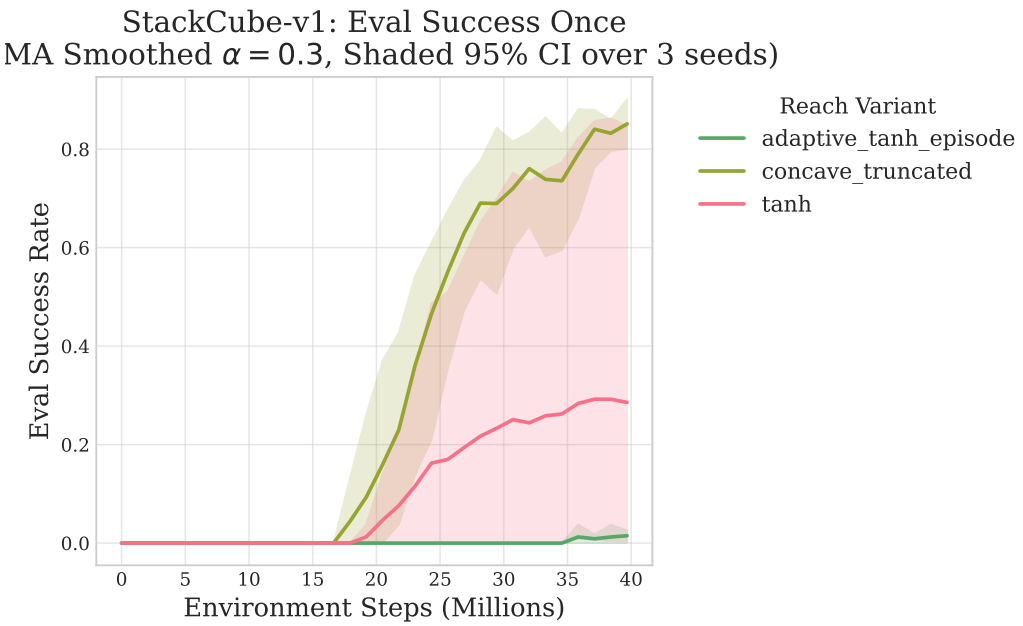

Figure 9: StackCube-v1: `eval/success_once` mean ± 95% CI. `concave_truncated` dominates; `adaptive_tanh_episode`—the best variant on all flat tasks—nearly completely fails here.

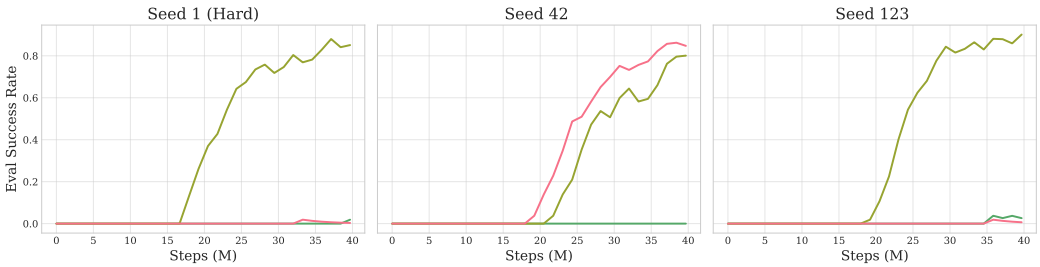

Figure 10: StackCube-v1 per-seed curves. `concave_truncated` learns reliably across all seeds. `adaptive_tanh_episode` is near-zero on seeds 42 and 123 where `tanh` recovers—a complete inversion of the PickCube ranking.

reaching reward stops competing with the downstream grasping/placement signal, allowing the policy to commit to contact. This is the inverse of the hover-attractor failure mode: here, reward *silence* near the goal creates a stable commitment region.

5.5 Summary: Task-Structure-Dependent Playbook

Table 5: Cross-task summary. Best variant per task in bold.

Task	**Type**	`tanh`	`concave_trunc.`	`adaptive_tanh_ep.`
PickCube-v1	Flat	0.625	0.469	**0.979**
PushCube-v1	Flat	0.854	—	**0.958**
LiftPegUpright-v1	Flat	0.792	—	**0.854**
StackCube-v1	Hierarchical	0.271	**0.896**	0.021

The results suggest a task-structure-dependent playbook:

- **Flat tasks** (PickCube, PushCube, LiftPegUpright): within-episode annealing (`adaptive_tanh_episode`) is optimal.
- **Hierarchical tasks** (StackCube): finite-support concave reward (`concave_truncated`) is optimal; time-based annealing is harmful.

However, this discrete playbook raises a deeper question: is there a *universal* reward function that works across both regimes? Section 6 describes `adaptive_tanh_stage`, our proposed answer.

6 Planned Experiments

6.1 Stage-Conditioned Annealing (`adaptive_tanh_stage`)

The StackCube inversion reveals that the failure of `adaptive_tanh_episode` is not fundamental to adaptive annealing—it is specific to scheduling k by *time* rather than *task progress*. We propose `adaptive_tanh_stage`: the same k schedule but with the clock reset at each stage transition.

Formally, let t_{stage} be the number of steps since the current task stage began (reset to 0 when the stage changes) and T be `max_episode_steps`:

$$k(t_{\text{stage}}) = k_{\min} + (k_{\max} - k_{\min}) \cdot \frac{t_{\text{stage}}}{T} \tag{2}$$

In StackCube, the stage transition is detected when `is_cubeA_grasped` flips. The moment the robot achieves a grasp, t_{stage} resets to 0 and the placement stage begins with $k{=}k_{\min}{=}2$ (wide), giving the policy a full exploration budget for the new sub-goal.

On single-stage tasks, $t_{\text{stage}} = t_{\text{elapsed}}$, so `adaptive_tanh_stage` is *identical* to `adaptive_tanh_episode`. This means it cannot regress on flat tasks—it can only improve StackCube.

Predicted outcome (Table **??**):

Table 6: Predicted results for `adaptive_tanh_stage`.

Task	`tanh`	`concave_trunc.`	`ep.`	`stage` **(predicted)**
PickCube	0.625	0.469	**0.979**	∼0.98
PushCube	0.854	—	**0.958**	∼0.96
LiftPegUpright	0.792	—	**0.854**	∼0.86
StackCube	0.271	**0.896**	0.021	>**0.85?**

Success criterion: StackCube mean > 0.85 across seeds $\{1, 42, 123\}$ with no regression on PickCube vs. `adaptive_tanh_episode` (0.979). If this holds, `adaptive_tanh_stage` becomes a universal reward function requiring only task-stage information already available in `evaluate()`.

6.2 PlaceSphere-v1 (Three-Stage)

PlaceSphere-v1 is a three-stage task (reach, grasp, place) with 13 reward components and the highest max reward of any task we study. It is the strongest test of whether stage-conditioned annealing generalises to deeper hierarchies. We plan 3 seeds, 5M steps, comparing `tanh`, `concave_truncated`, and `adaptive_tanh_stage`.

6.3 Gating and Terminal Dimensions

Fix the best reaching variant and sweep the gating and terminal bonus dimensions on PickCube-v1. Then cross-validate the top configurations on PushCube-v1, StackCube-v1, and PlaceSphere-v1.

6.4 Algorithm Generalization

Repeat the top configurations with SAC and a model-based method to assess whether the geometric findings are specific to on-policy optimization or more general.

6.5 Playbook and Sim-to-Real (Stretch)

Synthesize results into a decision-tree playbook. Validate the top configuration on the SO-101 robotic arm via the `lerobot-sim2real` bridge (already supported in ManiSkill3).

7 Discussion

Is the playbook a discrete decision tree or a continuous theory? The two-way split (flat: annealing; hierarchical: finite-support) is a *presentation* of a continuous underlying theory. The universal reward function $r(d, t_s) = 1 - \tanh(k(t_s) \cdot d)$, with k scheduled by stage progress rather than wall-clock time, subsumes both regimes. Flat tasks have one stage; hierarchical tasks have $n > 1$. The discrete playbook is the limit where this schedule is approximated by a static choice per task type.

Does the finding generalize beyond PPO? PPO's on-policy rollout may amplify hover behavior because the same near-goal trajectories are replayed within a batch. Off-policy methods or model-based approaches may be more or less susceptible to the failure modes observed here.

What is the effective zone radius? We used a 0.2m threshold for `concave_truncated`. This is a hyperparameter that should be tuned or set to reflect the task geometry (e.g., gripper reach relative to object size). We plan a sensitivity analysis.

Train/eval success gap as a reward quality metric. Explained variance is uninformative (uniformly ≈ 0.96), but the train/eval success gap may be a better proxy for reward hacking. We track this in Figure 6.

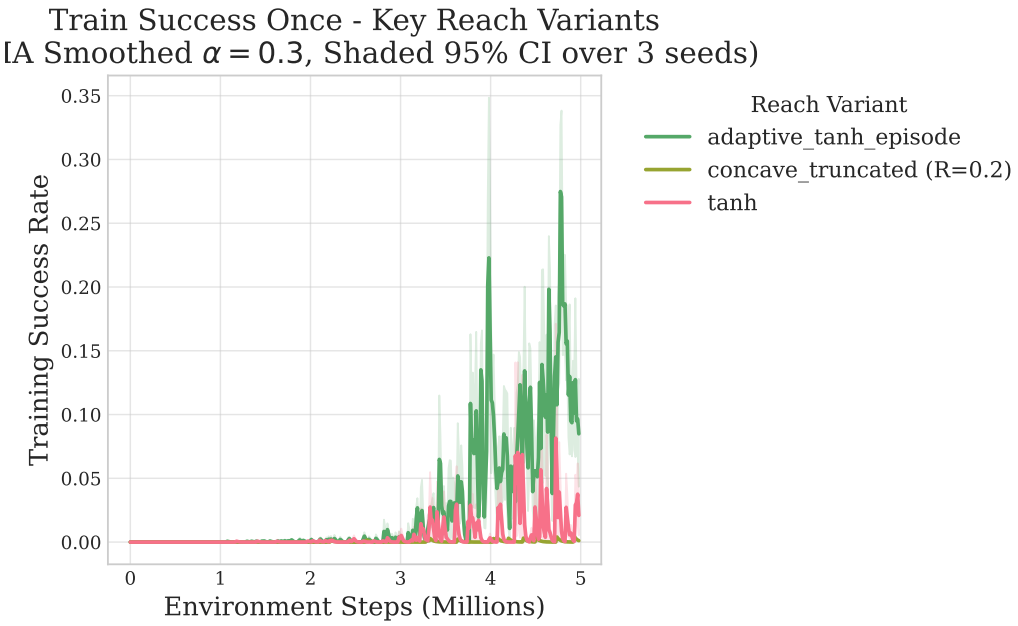

Figure 11: PickCube training success rates. Compare against Figure 1 (eval success) to observe the train/eval gap.

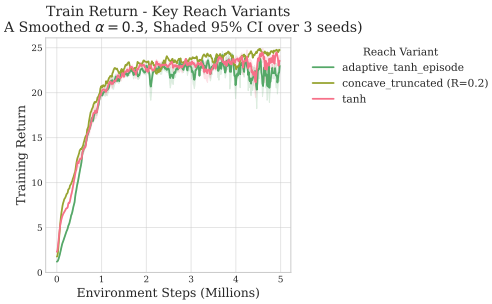

Figure 12: PickCube Training Returns

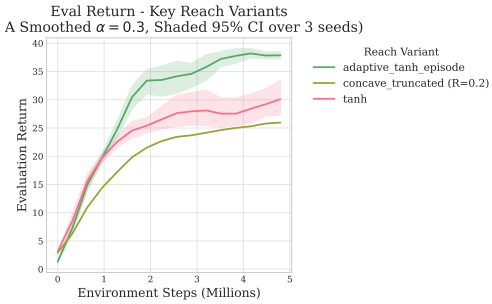

Figure 13: PickCube Evaluation Returns

8 Conclusion

We have presented the Reward Engineering Playbook: a systematic empirical study of how dense reward geometry causally affects manipulation learning. Across four tasks and three seeds per configuration, we identify a fundamental robustness-ceiling tradeoff between finite- and infinite-support reward geometry, show that within-episode scale annealing resolves this tradeoff on flat tasks while catastrophically failing on hierarchical ones, and propose stage-conditioned annealing as a principled fix. The complete ranking inversion on StackCube—from best (0.979) to worst (0.021) for `adaptive_tanh_episode`—is the key falsifying result that motivates the stage-conditioned theory. Experiments to validate `adaptive_tanh_stage` and extend to PlaceSphere-v1 are underway.

References

[1] Y. J. Ma, W. Liang, G. Shi, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[3] S. Devlin and D. Kudenko. Dynamic potential-based reward shaping. In *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 433–440, 2012.

[4] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Origo, P. Humphreys, O. Bachem, and M. Geist. What matters in on-policy reinforcement learning? a large-scale empirical study. In *International Conference on Machine Learning (ICML)*, pages 520–531. PMLR, 2021.

[5] J. Gu et al. Maniskill: A unified gpu-parallelized environment for robotics manipulation. *arXiv preprint*, 2024.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018.

A Supported Tasks and Stage Structure

Table **??** lists all 11 ManiSkill3 tabletop tasks instrumented with reward variants in this study, along with their episode length and stage structure. "Stage" refers to whether the task has a single committed reaching-and-contact phase (flat) or multiple committed contact stages with discrete transitions (hierarchical). On single-stage tasks, `adaptive_tanh_stage` is mathematically identical to `adaptive_tanh_episode`.

Table 7: ManiSkill3 tasks with reward variant support.

Task	**Stage structure**	**Max steps**	**Training steps**
PickCube-v1	Single (reach, grasp, lift)	50	5M
PushCube-v1	Single (reach push, push to goal)	50	5M
LiftPegUpright-v1	Single (reach, grasp, orient)	50	5M
PullCube-v1	Single (reach, pull to goal)	50	5M
PokeCube-v1	Single (reach, poke to position)	50	5M
RollBall-v1	Single (reach, roll to goal)	80	5M
PickSingleYCB-v1	Single (reach, grasp, lift)	50	5M
PlaceSphere-v1	Hierarchical (reach, grasp, place)	50	5M
PullCubeTool-v1	Hierarchical (grasp tool, pull cube)	100	5M
StackCube-v1	Hierarchical (grasp, stack)	50	20M
PegInsertionSide-v1	Hierarchical (reach, grasp, orient, insert)	100	**[TODO: TBD]**